



SATQUERY AI - FINAL MASTER BLUEPRINT

The Complete, Consolidated, Build-Ready Plan

All three analyses agree on the core truth: this is winnable, but only if you build a real agentic system with real RS adaptation – not a chatbot wrapper. Here is the single unified plan.



THE CORE MENTAL MODEL (Lock This In)

text

WRONG MENTAL MODEL:

"We're building a satellite chatbot"

RIGHT MENTAL MODEL:

"We're building a miniature Remote-Sensing Operating System where an agentic controller routes between specialist models and returns auditable, evidence-grounded answers"

THE CHATBOT IS THE FRONT DOOR.

THE REAL PRODUCT IS THE ORCHESTRATION LAYER.



THE ONE SENTENCE THAT WINS THIS PS

text

"The controller may perform internal task planning; however, only the observable execution trace... will be evaluated."

TRANSLATION:

You don't need to beat published SOTA benchmarks.

You need a system that VISIBLY and CORRECTLY:

- Classifies the task
- Validates the input
- Selects the right tool
- Shows its work
- Returns grounded evidence

This is an ENGINEERING problem as much as an ML problem. That makes it tractable.



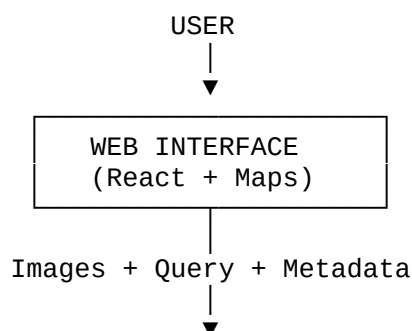
PART 1: REQUIREMENT → MODEL MAPPING (LOCKED)

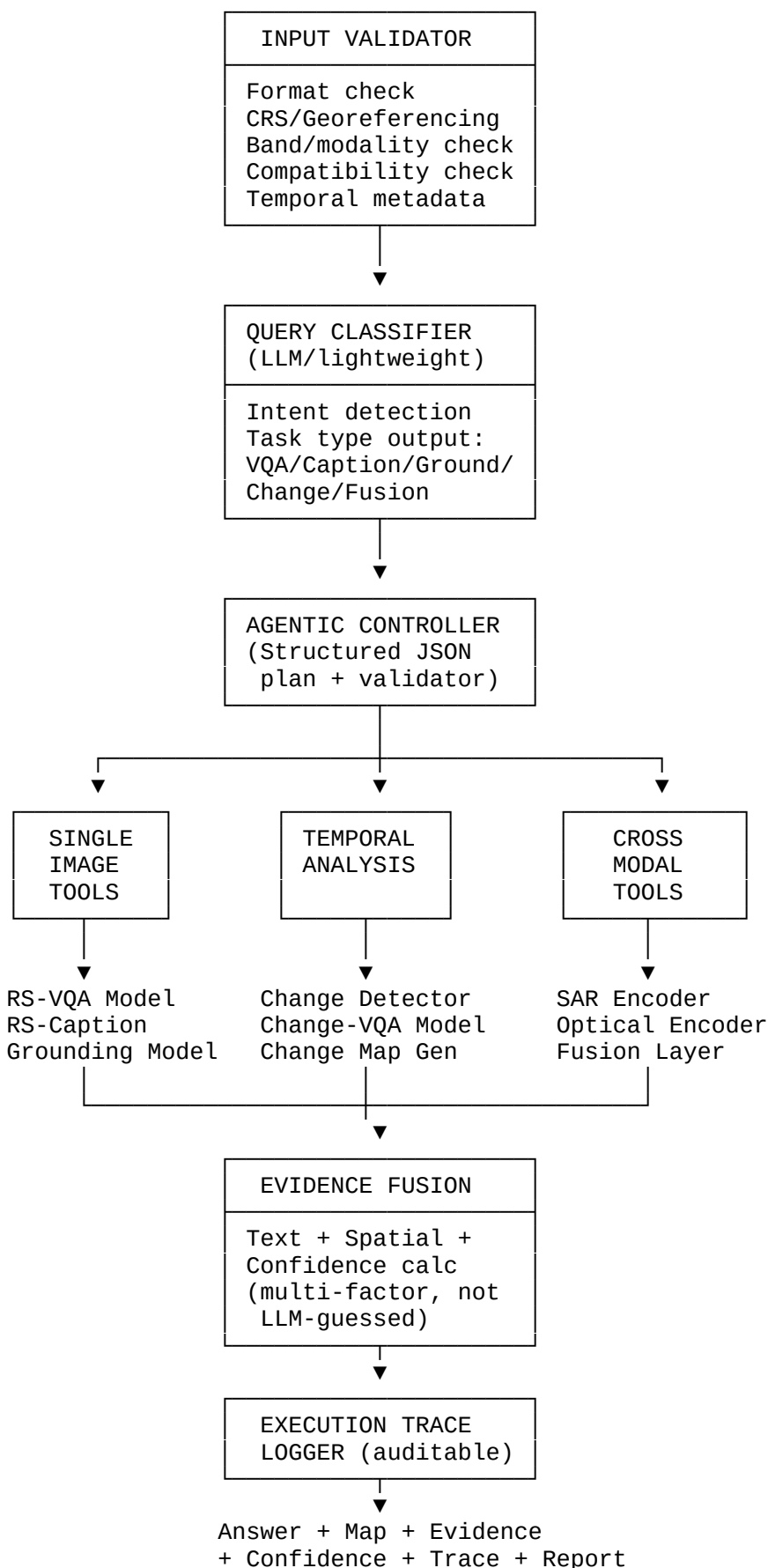
PS Requirement	Model/Approach	Dataset
RS Adaptation (mandatory)	LoRA fine-tune on base RS-VLM	BigEarthNet.txt
Single-image VQA (mandatory)	Fine-tuned RS-VLM RSVQA, VRSBench	
Captioning OR Grounding Do BOTH for extra credit	VRSBench	
Bi-temporal change (mandatory)	Change detector + Change-VQA head	CDVQA, LEVIR-CD
Optical-SAR fusion (mandatory)	Dual-encoder + cross-attention	BigEarthNet.txt (S1+S2)
Agentic orchestration (mandatory) (your code)	Deterministic router + tool registry	N/A
GeoTIFF support (mandatory)	rasterio + GDAL	N/A



PART 2: FINAL SYSTEM ARCHITECTURE

text





🧠 PART 3: THE AGENT DESIGN PRINCIPLE (CRITICAL) text

DO NOT let the LLM be the scientist.
DO NOT let the LLM freely pick arbitrary tools.

WRONG:

Image → LLM → Answer (LLM invents numbers)

RIGHT:

LLM = Classifier + Explainer only

ML Models = Actual computation

FLOW:

Query → LLM classifies intent → Structured JSON plan
→ Policy Validator (checks allowed tools/params)
→ Execute ONLY validated tools
→ Real models compute real numbers
→ LLM verbalizes the computed result

Example JSON plan:

```
{
  "task": "change_vqa",
  "inputs": ["image_t1", "image_t2"],
  "tools": ["change_detector", "change_vqa_head"]
}
```

Validator checks:

- ✓ change_vqa allowed
- ✓ requires two images ✓ present
- ✓ temporal pair detected ✓
- ✓ parameters within policy
- ONLY THEN execute

This gives you a CONTROLLED agent,
not a hallucinating one. This is the
single most important architectural
decision in the whole project.

📁 PART 4: MODEL REGISTRY (Modular, Pluggable)

Python

```
MODEL_REGISTRY = {
    "vqa": {
        "model": "RS-VLM (GeoChat-based, LoRA fine-tuned)",
        "input": "single_image + question",
        "output": "answer + confidence"
    },
    "caption": {
        "model": "RS-Captioner (same base, different head)",
        "input": "single_image",
        "output": "scene description"
    },
    "grounding": {
        "model": "RSVG / GroundingDINO fine-tuned on RS classes",
        "input": "image + text query",
        "output": "bbox / mask"
    },
    "change_detection": {
        "model": "ChangeFormer / Siamese U-Net",
        "input": "image_pair",
        "output": "change_map + stats"
    },
    "change_vqa": {
        "model": "Fine-tuned on CDVQA",
        "input": "image_pair + change_map + question",
        "output": "answer + confidence"
    },
    "optical_sar_fusion": {
        "model": "Dual-encoder + cross-attention",
```

```

        "input": "optical + SAR (co-registered)",
        "output": "joint analysis + segmentation"
    }
}

```

 PART 5: CONFIDENCE – DO IT PROPERLY
text

DO NOT: "LLM says 92%" ← meaningless

DO: Multi-factor calibrated confidence

```

Confidence = f(
    model_confidence,
    specialist_agreement,
    input_quality (cloud cover, resolution),
    spatial_consistency,
    temporal_consistency
)

```

Thresholds (calibrate experimentally):

```

HIGH    > 0.85
MEDIUM 0.60 - 0.85
LOW     < 0.60

```

Show WHY confidence is what it is:

"Confidence: 91% – SAR confirms structure
that optical shows partially obscured by cloud"

This single detail signals scientific
maturity to ISRO judges.

 PART 6: THE THREE KILLER DEMOS (Not 25 Features)
text

DEMO 1: Single Image Intelligence

Upload: One optical image

Query: "Describe the land cover and highlight
the water body"

Shows: VQA + Caption + Grounding running together

Output: Answer + caption + bounding box + confidence

DEMO 2: Temporal Change Intelligence (MAIN DEMO)

Upload: T1 (2019) + T2 (2024) of same location

Query: "What changed and where?"

Shows: Change detector → change map → Change-VQA

Output: Before/After/Change-map + description + %

Follow-up query (reuses previous analysis):

"Has built-up area increased?"

→ System reuses computed change map

→ Feels genuinely agentic, not restarting blind

DEMO 3: Cross-Modal Fusion (Differentiator)

Upload: Co-registered Optical + SAR pair

Query: "Use both images to identify built-up
and water-covered regions"

Shows: SAR encoder + Optical encoder + fusion

Output: Combined segmentation map (better than
either alone – SAR sees through the
clouds optical missed)

THIS IS THE DEMO THAT PROVES YOU DID NOT

FAKE THE HARDEST REQUIREMENT.
🖥️ PART 7: UI LAYOUT (Locked Design)
text

SATQUERY AI

IMAGE INPUT

Image T1

Image T2

[Optional]

Modality: ☐ Optical ☐ SAR ☐ Both

Query: [Has the built-up area increased?]

TASK: CHANGE VQA

EXECUTION TRACE

- ✓ Input validated
- ✓ Query classified
- ✓ Change detector run
- ✓ Change-VQA executed
- ✓ Confidence calibrated

VISUAL EVIDENCE

[Before]

[After]

[Change Map]

ANSWER: Built-up area increased ~12.7%

Confidence: 91% (HIGH)

Models used: ChangeFormer, GeoChat-CDVQA

[Download Full Report]

🔗 PART 8: EVALUATION HARNESS (Build This – It's a Differentiator)
text

Most teams show a pretty UI with no numbers.
You show REAL benchmark scores.

```
evaluation/  
  vrsbench/  
  rsvqa/  
  cdvqa/  
  bigearthnet/  
  custom_isro_simulation/
```

Command: python evaluate.py --benchmark all

Output shown to judges:

```
=====
SATQUERY AI EVALUATION
=====
VRSBench    → VQA: 78.4  Caption: 81.2  Ground: 74.6
RSVQA       → VQA: 83.1
CDVQA       → Change-VQA: 71.8
Cross-modal → Optical-SAR: 79.3
-----
Overall: 80.1
=====
```

Also build a FAILURE ANALYSIS dashboard:
Correct: 81.3% | Incorrect: 12.4% | Uncertain: 6.3%

Failure breakdown:
SAR interpretation: 28% | Small objects: 21%
Temporal ambiguity: 18% | Cloud contamination: 13%

This demonstrates engineering maturity that 95% of competing teams will not show.

🚀 PART 9: HIDDEN ISRO DATASET STRATEGY

text

You will NEVER see Cartosat-2S/RISAT eval labels.

DO NOT overfit to Sentinel-2 → Sentinel-2 only.

DESIGN FOR SENSOR-INDEPENDENCE:

Raw Image (any sensor)

↓

Sensor-agnostic preprocessing

```
(normalize dynamic range, resample resolution,
handle band count differences)
```

↓

Common feature representation

Task-specific heads

Your pipeline must tolerate:

- Different spatial resolutions

→ Different band counts

- Different dynamic ranges

- SAR speckle noise variations

- Missing/partial metadata

- Different CRS

This generalization mindset matters more than chasing SOTA on public benchmarks.

PART 10: FINAL REPOSITORY STRUCTURE

text

satquery-ai/

```
├── frontend/ # React app
│   ├── src/components/
│   ├── src/pages/
│   └── src/utils/leaflet-map/
├── backend/
│   ├── api/ # FastAPI endpoints
│   ├── controllers/ # Agentic controller
│   ├── validators/ # Input/policy validation
│   ├── planners/ # Query→JSON plan (LLM)
│   ├── fusion/ # Evidence fusion
│   └── reporting/ # PDF report generator
├── models/
│   ├── vqa/
│   ├── caption/
│   ├── grounding/
│   ├── change/
│   └── optical_sar/
├── training/ # Fine-tuning scripts
│   ├── bigearthnet/
│   ├── vrsbench/
│   ├── rsvqa/
│   └── cdvqa/
├── geospatial/
│   ├── raster.py # rasterio wrapper
│   └── metadata.py
```

```

├── reprojection.py
├── registration.py           # co-registration checks
├── agent/
│   ├── router.py
│   ├── registry.py          # MODEL_REGISTRY
│   ├── policy.py            # allowed tools/params
│   ├── executor.py
│   └── trace.py             # audit log generator
├── evaluation/
│   ├── vrsbench/
│   ├── rsvqa/
│   ├── cdvqa/
│   └── custom_isro_sim/
├── data/
├── configs/
├── tests/
├── docs/
├── docker/
└── README.md

```

 PART 11: TEAM STRUCTURE (6 Members, Locked Roles)
text

PERSON 1 (Yash) – Agent/Backend Architect
→ Controller, router, model registry, execution trace
→ LangGraph/structured JSON planning
→ THIS IS YOUR STRONGEST SKILL – own it fully

PERSON 2 – ML/VLM Engineer
→ RS-VLM selection + LoRA fine-tuning
→ BigEarthNet training pipeline
→ VQA + Captioning heads

PERSON 3 – Computer Vision Engineer
→ Change detection model (ChangeFormer)
→ Grounding model
→ Change-VQA head

PERSON 4 – Geospatial Engineer (CRITICAL HIRE)
→ GeoTIFF/rasterio/GDAL
→ CRS handling, co-registration checks
→ Metadata extraction
→ MOST IMPORTANT PERSON TO FIND

PERSON 5 – Frontend/Product Engineer
→ React dashboard, Leaflet maps
→ Evidence visualization (overlays, masks)
→ Execution trace UI panel

PERSON 6 – Evaluation/Research + Presentation
→ Benchmark harness (VRSBench/RSVQA/CDVQA)
→ Failure analysis dashboard
→ Documentation + pitch deck + demo scripting

 PART 12: PHASED DEVELOPMENT ROADMAP
text

PHASE 0 – Understand (Week 1)
PS → requirements matrix → dataset matrix
→ model matrix → evaluation matrix

PHASE 1 – Baseline (Week 2-3)
GeoTIFF → VQA → Answer (single image working)

T1+T2 → Change detection → Answer (basic)

PHASE 2 – Specialist System (Week 4-5)
Add: Captioning, Grounding, Change-VQA,
Optical-SAR fusion

PHASE 3 – Agent Layer (Week 6)
Query → Intent classifier → Tool selection
→ Execution → Trace

PHASE 4 – Evidence Layer (Week 7)
Bounding boxes, masks, change maps, overlays,
confidence calibration

PHASE 5 – Adaptation (Week 8-9)
Fine-tune on BigEarthNet.txt (LoRA)
Evaluate on VRSBench, RSVQA, CDVQA
Document loss curves + before/after comparison

PHASE 6 – Robustness (Week 10)
Test: different resolutions, sensors, sizes,
clouds, SAR noise, missing metadata

PHASE 7 – ISRO Simulation (Week 11)
Build your OWN hidden-test-like set using
Indian terrain + optical + SAR + temporal pairs
(Don't just test on public benchmarks)

HACKATHON (36 hrs) – Integration + Polish + Demo
✅ PART 13: "DEFINITION OF DONE" CHECKLIST
text

CORE

- ☐ GeoTIFF + TIFF upload working
- ☐ Optical vs SAR auto-detection
- ☐ Metadata extraction (CRS, bands, resolution)
- ☐ Compatibility checking with clear errors
- ☐ Single-image VQA
- ☐ Captioning
- ☐ Grounding
- ☐ Bi-temporal change detection + change map
- ☐ Change-VQA
- ☐ Optical-SAR fusion analysis

AI

- ☐ Remote-sensing adaptation (LoRA on BigEarthNet)
- ☐ Benchmark evaluation numbers documented
- ☐ Confidence calibration (multi-factor, not LLM guess)
- ☐ Failure handling / graceful degradation

AGENT

- ☐ Query classification working
- ☐ Structured JSON plan + policy validator
- ☐ Tool registry pluggable
- ☐ Execution trace visible in UI
- ☐ Result fusion (text + spatial + confidence)

UI

- ☐ Image viewer + map
- ☐ Before/after slider
- ☐ Masks/bounding box overlays
- ☐ Confidence display
- ☐ Execution trace panel
- ☐ Report download (PDF)

ENGINEERING

- Docker containerization
- Reproducible setup (README)
- Error handling throughout
- Offline/local demo capability (no internet dependency)

🔧 PART 14: THE WINNING PITCH STRUCTURE
text

[Opening – NOT "we built an AI"]
"Today, extracting one reliable answer from satellite imagery requires a remote-sensing expert, GIS software, and multiple specialized tools. We built SatQuery AI to turn that workflow into a single natural-language conversation – grounded in real evidence, not guesses."

[Differentiation line]
"Unlike a generic vision-language model – which this problem statement explicitly says cannot satisfy the requirement – SatQuery never trusts one model. It identifies intent, validates the imagery, routes to the right domain specialist, cross-checks the result, and shows you exactly how it got there."

[Live Demo – 3 scenarios in order]
Demo 1: Single image VQA + grounding
Demo 2: Bi-temporal change (MAIN – most impressive)
Demo 3: Optical-SAR fusion (the hardest requirement, proven working – not faked)

[Evidence of rigor]
"Here are our benchmark scores on VRSBench, RSVQA, and CDVQA – and here's our fine-tuning loss curve showing real adaptation on BigEarthNet, not a generic prompt wrapper."

[Close]
"This isn't a chatbot for satellites. It's a foundation for how ISRO's data becomes usable by every district officer, farmer, and researcher in India."
⚠️ PART 15: TRAPS THAT SINK OTHER TEAMS (Avoid All)
text

x Generic VLM with no fine-tuning evidence → disqualified
x Faking optical-SAR fusion with a text template
x Only accepting PNG/JPEG, no real GeoTIFF handling
x Black-box agent with no visible execution trace
x LLM inventing numbers instead of using computed values
x Fake/uncalibrated confidence scores
x Demoing 25 half-working features instead of 3 solid ones
x Training a giant VLM from scratch (waste of time)
x Ignoring SAR entirely (PS explicitly emphasizes it)
x Starting data wrangling during the hackathon itself
x Claiming "fine-tuning" when you only prompted a VLM
🏆 PART 16: WHERE YOUR WINNING MARGIN COMES FROM
text

Most competing teams (predicted profile):
UI ██████████ (strong)
LLM wrapper ██████████

Agent		
Remote sensing		(weak – most skip this)
SAR		(weak – most fake this)
Evaluation		(weak – most show no numbers)

YOUR TARGET PROFILE:

UI			
Remote sensing			
VLM (fine-tuned)			
Change analysis			
Optical-SAR fusion			← differentiator
Agent orchestration			← your core strength
Evaluation/benchmarks			← differentiator

The gap is exactly where most teams are weak:
 real RS adaptation, real SAR fusion, real numbers.

🚀 IMMEDIATE NEXT ACTIONS (This Week)
 text

1. Find Person 4 (Geospatial/RS expert) – HIGHEST PRIORITY
 2. Download BigEarthNet.txt + VRSBench + RSVQA + CDVQA
 3. Set up GDAL/rasterio environment (GDAL via conda FIRST)
 4. Pick base RS-VLM (GeoChat or similar) and run baseline inference
 5. Build the agent skeleton (router + registry + trace logger)
 - this can start immediately, no GPU needed
 6. Draft the requirement → feature → model matrix as a shared doc
 7. Assign the 6 roles above to your actual team members
- This is the complete, consolidated blueprint. Every suggestion from all three analyses is merged into one coherent build plan.